

Design and Implementation of Full-Scale Industrial Control System Test Bed for Assessing Cyber-Security Defenses

Robert E. Gillen^{*}, Laura Ann Anderson^{*}, Christopher Craig^{*}, Jordan Johnson^{*},
Adam Columbia[†], Rachel Anderson[‡], Andrew Craig[§] and Stephen L. Scott[§]

^{*}Oak Ridge National Laboratory

[†]University of Texas, San Antonio

[‡]Massachusetts Institute of Technology

[§]Tennessee Technological University

Abstract—In response to the increasing awareness of the Ethernet-based threat surface of industrial control systems (ICS), both the research and commercial communities are responding with ICS-specific security solutions. Unfortunately, many of the properties of ICS environments that contribute to the extent of this threat surface (e.g. age of devices, inability or unwillingness to patch, criticality of the system) similarly prevent the proper testing and evaluation of these security solutions. Production environments are often too fragile to introduce unvetted technology and most organizations lack test environments that are sufficiently consistent with production to yield actionable results. Cost and space requirements prevent the creation of mirrored physical environments leading many to look towards simulation or virtualization. Examples in literature provide various approaches to building ICS test beds, though most of these suffer from a lack of realism due to contrived scenarios, synthetic data and other compromises. In this paper, we provide a design methodology for building highly realistic ICS test beds for validating cybersecurity defenses. We then apply that methodology to the design and building of a specific test bed and describe the results and experimental use cases.

Index Terms—industrial control systems, test beds, cyber security

I. INTRODUCTION

In response to the increasing awareness of the Ethernet-based threat surface of industrial control systems (ICS) [1], both the research and commercial communities are responding with ICS-specific security solutions [2]–[6]. Unfortunately, many of the properties of ICS environments that contribute to the extent of this threat surface (e.g., age of devices, inability or unwillingness to patch, criticality of the system [1], [7]) also prevent the proper testing and evaluation of these security solutions. Production environments are often too fragile to introduce unvetted technology and most organizations lack test environments that closely mirror production environments to

yield actionable results. Cost and space requirements prevent the creation of mirrored physical environments leading many to look towards simulation or virtualization.

Various approaches to building ICS test beds have been proposed, though most of these suffer from a lack of realism due to contrived scenarios, synthetic data and other compromises [8]. Depending on the intent of the test bed, these compromises may negatively effect the results. As an illustration, the commonly-held notion that ICS network traffic is highly regular was challenged via a longitudinal study by Formby et al. in [9]. Therefore, evaluating certain network-based anomaly detection approaches in a test environment exhibiting highly regular traffic could produce misleading results.

In this paper, we provide a design methodology for building ICS test beds to simplify the evaluation of network-based intrusion detection systems (IDS) or similar network security approaches. To obtain an appropriate understanding of how these systems will operate in the production environment, the test environment should be behaviorally consistent with the systems of interest. Behavioral consistency includes the steady-state behavior of devices in the environment, their responses to various stimuli and the characteristics of the network (e.g., packet rate, data rate, protocol distribution). Our approach

- 1) was developed based on a realistic scenario,
- 2) produces realistic network traffic, and
- 3) demonstrates an approach for realistically mirroring production and test bed environments.

The remainder of this paper is organized as follows. In the next section we discuss related research and ICS lab design approaches and classifications. In Section 3 we describe our methodology. Section 4 details a concretization of the methodology and applies them to a specific use case. Section 5 describes the validation steps for the test bed to confirm it exhibits behavioral consistency and Section 6 concludes the paper and discusses our future research plans.

This manuscript has been co-authored by UT-Battelle, LLC under Contract No. DE-AC05-00OR22725 with the US DOE. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this manuscript, or allow others to do so, for United States Government purposes. The DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

II. RELATED WORK

In lieu of an exhaustive survey of the ICS test beds represented in the literature, we provide an overview of a few surveys and then describe a representative set of examples that highlight the unique aspects of our design and method.

A. Test Bed Surveys

Holm et al. provide a survey of 30 ICS test beds in [8]. They highlight that in the majority of the reviewed cases, even though the stated aim was to facilitate vulnerability analysis the test beds rely heavily on simulation models; the topic of fidelity (relative to production environments) is rarely addressed (not at all in 68%, experientially in 26% and quantifiably so only in 6%). One problem in using a model-based simulation for vulnerability assessment is programmable logic controller (PLC) crashes; [10] showed that 18% of the tested PLCs crashed as a result of a simple vulnerability scan. The faulty behavior that causes these crashes must be reproduced in the simulated versions to achieve behavioral consistency and prevent incomplete results.

In [11] the authors provide a survey of various implementation approaches seen in ICS test beds and provide a qualitative ranking of the suitability of each of these approaches across various metrics of interest. Their list of approaches include physical replication, simulated, virtual, virtual-physical, and hybrid while their metrics of interest are fidelity, repeatability, accuracy, safety, cost-effectiveness, reliability and scalability. Their measurements are somewhat subjective; however their method provides an evaluation of the strengths and weaknesses of each approach. Our approach falls under the virtual-physical class which is scored as “high” for fidelity and “excellent” for accuracy while receiving “low” and “moderate” scores for cost-effectiveness and scalability.

B. Test Bed Examples

In [1] the National Institute of Standards and Technology (NIST) describes a test bed they intend to build that supports at least three different scenarios: chemical and robotic assembly plants, and wide-scale distribution networks such as oil or gas pipelines. They propose an environment that is re-configurable and uses simulation tools such as MatLab’s Simulink for various aspects of the platform such as communication delays due to bandwidth-constrained devices. While the NIST environment will support a broad range of scenarios, it does at the cost of behavioral consistency (e.g., model based, simulation-based hardware).

The design philosophy of the National Supervisory Control and Data Access (SCADA) Testbed [12] is similar to ours. It makes heavy use of physical equipment and can be used to accurately assess network-reachable vulnerabilities in various ICS devices. The physical scale of that test bed renders it both fiscally and practically infeasible for many organizations. Additionally, certain test cases may destroy expensive equipment, thereby excluding them from research plans. For example, the potential destruction of one full-scale transformer may

preclude a series of experiments that would otherwise provide valuable insight.

Ghassempour et al. describe a hardware-in-the-loop (HIL)-based test bed they developed for the University of South Florida [13]. This platform facilitates experiments involving various power management schemes, grid cyber attacks and their mitigations. Their support for HIL allows them to dynamically reconfigure the test bed to use real hardware or simulate that hardware using Labview and custom-developed integrations. Their use of simulated SCADA devices differs from ours; their use of captures from physical systems is similar to ours.

A SCADA test bed specifically focused on cyber-security and forensic research is presented in [14]. The authors use a combination of fictitious (yet inspired from real life) scenarios and physical hardware such as PLCs, transformers, and pumps. Similar to our approach, this environment can serve to test network protocols and hardware vulnerabilities. Because this system does not have a physical counterpart we cannot determine its behavioral consistency.

The Secure Water Treatment Test bed (SWaT) [15] designed and built by the Singapore University of Technology and Design is a fully-physical ICS test bed intended to support different modalities of cyber security testing. In SWaT, they reproduce a small-scale six-stage water treatment plant using physical devices for both the ICS as well as all of the sensors and actuators. Their environment enables vulnerability testing of equipment and wired and wireless network protocols. Unlike our approach, their scenario, while informed by a real-world water treatment facility, is custom built for their laboratory as are the PLC, human-machine interface (HMI) and historian software configurations. These configurations risk introducing behavior that is inconsistent with their target facility. However, their approach may represent the broad class of water treatment facilities better.

III. DESIGN OBJECTIVES

Test bed design is an exercise in compromise. Many of the most desirable characteristics of a test platform (e.g., reconfigurability, generalizability, accuracy) are diametrically opposed to each other. Our specific objective is to design a validation laboratory for network-based intrusion detection systems that are being considered for deployment to a particular production environment. Our hypothesis is that in order to accurately assess a network-based intrusion detection device or similar network protection system, the test environment must closely resemble the target environment. What follows is a list of our design objectives and the compromises they introduce.

A. Scenario Realism

Objective 1: Where possible, the scenario embodied in the ICS test bed should mirror the target as closely as possible. This requires coordination and information sharing with the designers and operators of the target environment. Ideally, the test environment will reproduce all devices (either in hardware or emulation) from the target and will have them similarly

configured and interconnected. Adherence to this objective may limit the flexibility or reconfigurability of the test bed.

B. Network Realism

Objective 2: The test bed network should exhibit behavior consistent with the target network. To accomplish this, a thorough and complete understanding of the target network environment is required. This includes a list of connected devices, the protocols they utilize and a characterization of the network(s) under consideration (e.g., packet count, average packet size, packet rate and protocol distribution). Where possible, full packet capture and subsequent analysis can provide the most accurate data source to ensure this characterization is faithful. Often, there exist devices or behaviours that would be inadvertently missed during an expert-knowledge interview but show clearly in the data.

C. Vulnerability Realism

Objective 3: For the specific devices of interest, any vulnerabilities that exist in the production version must be faithfully reproduced in the test bed. Many test beds discussed in the literature simulate or virtualize ICS devices. Both approaches have obvious benefits including scalability, reduced cost and reconfigurability. Due to the disparity between specification and implementation combined with nuances introduced during the manufacturing process, there often exists vulnerabilities in physical devices that do not appear in a specification-based virtualization or simulation. Further, the process of virtualization or simulation may introduce vulnerabilities not found in the physical hardware. Consideration must be given to the nature of the vulnerabilities the test bed is intended to uncover. Once this exercise has been completed, decisions regarding which components can be simulated and which should be represented in hardware become straight-forward.

D. Achievable Failure

Objective 4: The environment must be able to reproduce device and system failure consistent with production. A key aspect to understanding the impact of a deficiency in an IDS or other network protection device is a clear picture of the cascading effects it may have. In an ICS environment, this includes the second and third order effects on the process, the equipment and safety. This objective ensures that the test bed can represent the “worst case scenarios”. Understanding these scenarios can be invaluable in communicating to others the relative severity of one configuration or vulnerability to another. Successfully achieving this objective requires an understanding of the failure and recovery mode(s) for each device. For some devices a simple factory reset or updated firmware image will restore it to normal operations. Others may have physical or environmental effects that are costly. Device cost and their failure mode will drive the way we represent devices in our system.

E. Safety

Objective 5: Complete system failure, even to include physical destruction of components, can be performed in a safe and responsible fashion. To achieve this, safety should be an overarching consideration in each of the above design stages. As with prior objectives, the safety objective may influence how we represent devices in our system. Safety introduces a natural tension with and between the prior objectives. Documenting these trade-offs is vital to understanding how these decisions affect the experimental realism produced when using our test bed.

IV. IMPLEMENTATION

Our exemplar ICS is the cooling system for Oak Ridge National Laboratory’s 200-petaflop Summit supercomputer, currently the fastest open-science computer in the world [16], [17]. This computer consists of over 4,600 nodes and has a peak power draw of 13MW. The infrastructure needed to cool Summit cycles through over 4,000 gallons of water each minute. Our test bed is a near duplication of the industrial controls supporting this cooling system. The mechanical engineering team that supports the National Center for Computational Sciences provided us with the design, implementation and specification documents for this system. Working from the “as-built” mechanical drawings, we designed a test bed that is logically divided into two parts: (a) control/monitoring devices and (b) sensors/actuators.

The methodology employed for the implementation of this test bed and those with similar objectives is:

- 1) To the extent possible, utilize duplicate devices
- 2) Record key communications and interactions. In the test bed, replay those in a realistic manner
- 3) When emulating or virtualizing devices, build them such that they can connect to the hardware devices using physical wires and communication channels
- 4) Construct and layout the test bed to closely mirror the design documents
- 5) Collaborate with the installers and operators of the original system

The implementation details below illustrate the application of this methodology and how it integrates the design objectives from Section 3. We link each section to the design objectives and methodology steps addressed therein in the form of [D:*x*, *y*; M:*z*] which would indicate a linkage to design objective *x* and *y* and methodology step *z*.

A. Control/Monitoring Devices [D:1, 2, 3; M: 4]

Our test bed uses the same physical equipment as our exemplar system. The controller is an Allen Bradley ControlLogix PLC with 34 I/O modules distributed over six chassis connected with an Ethernet/IP ring-topology backbone. The HMI is an Allen Bradley PanelView Plus series device and the historian is a Yokogawa data acquisition unit. We also used identical industrial Ethernet switches and 24 VDC power supplies. We connected an engineering workstation equipped with software for viewing and configuring the historian as well



Fig. 1. Control and Monitoring devices portion of the test bed. These devices are an exact match to the configuration used in the production environment. Keeping them identical allows for direct deployment of production code as well as accurate assessments of their robustness to various attacks.

as the PLC and HMI (Rockwell Studio 5000). This portion of the test bed can be seen in figure 1.

B. Sensors/Actuators [D: 4, 5; M: 3, 4]

Over 500 sensors and actuators provide data to the PLC and respond to its commands. Most of these sensors interact with the PLC via hard-wire electrical signals such as 4mA-10mA analog signals, 0/20 VDC digital signals, or 100/1000 ohm resistance temperature detectors (RTDs). The primary use for our test bed is evaluating security devices connected to the Ethernet-based network. This allows us to emulate all of the physical sensors and actuators assuming they can be configured to provide identical inputs/outputs (compared to the production counterparts) to the PLC. The fact that they are emulated does not affect the behavior of the Ethernet network in the experimental environment.

Due to the physical size and cost constraints, we emulated these using over 40 Raspberry Pis and approximately 200 daughter boards. Each Pi is responsible for a logical grouping of sensors/actuators based on the mechanical drawings (e.g., sensors and valves from a given cooling tower are grouped). The majority of the daughter boards are connected to the Pis via serial peripheral interface (SPI) and provide the analog in, analog out, and digital in signals. Standard relay boards are connected via general purpose input/output (GPIO) pins and provide digital out signals. Our custom-built RTD emulators are connected via I2C. To the PLC, these devices are indistinguishable from their real-world counterparts. Figure 2 shows this portion of the test bed.

1) *Variable Frequency Drives (VFDs)*: While the majority of these devices are line-level signals as described above, a number of the VFDs utilize an Ethernet-based signal line in addition to their line-level signals. In the production environment, this Ethernet connection is used for general trend and state data. We are able to capture raw traffic from the production environment, we modeled in software the communication protocol, traffic rate, and VFD-to-PLC handshakes which are standard common industrial protocol (CIP) packets. We deployed a VFD emulation script on the appropriate Pis and confirmed their communications with the PLC.



Fig. 2. Portion of test bed representing the sensors and actuators in the physical plant. Each screen represents a logical grouping of sensors and is designed to represent “ground truth” during experiments wherein the human machine interface (HMI) may be compromised and showing inaccurate data.

2) *Experimental Support*: One disadvantage to emulating the physical sensors and actuators is that during an experiment wherein the adversary’s objective is loss-of-view (e.g., HMI is disabled or its data is incorrect), you have no way of visually knowing that what the HMI is displaying is incorrect. To address this issue, Figure 2 illustrates that each grouping of sensors has a display that is independent of the operational technology (OT) network and is directly connected to those sensors. When the emulation software sets signal values for each of its sensors, it also updates the attached screen. This allows a user to see the “physical” state of a given valve as closed even though the HMI and historian may be reporting it as open.

C. Hardware Device Configurations [D: 1; M: 1, 5]

We configured each of the physical hardware devices with the exact configuration files used in production. The historian, HMI, and network switches accepted the production configuration with no modifications. The ladder logic deployed to the PLC required minor changes due to environmental differences between the production environment and the test bed. Specifically, the production environment has some logic for connectivity to higher-level facility management systems as well as a fully-redundant PLC. These features are absent in our test bed.

D. Replay of Production Data [D: 1, 2; M: 2]

To provide a high-level of consistency with the actual physical plant, we obtained 30-days worth of data from the production historian. This data was then parsed and split into the logical sensor groups represented by each Pi/emulation platform. An emulation script was developed that processes this data and transmits/receives real signals to/from the PLC for each sensor at a rate of 2 Hz. The historian in the production environment was configured for a much lower sampling rate, so the emulator needs to interpolate between sample points. A centralized agent on a network isolated from the OT ensures that all of the emulation devices are synchronized. This configuration yields a simple and consistent “model” without requiring complicated physics-based models or other intensive computations to occur.

V. VALIDATION

This section describes the validation steps taken to confirm that the test bed built in Section 4 meets the objectives described in Section 3.

A. Comparisons

We validated the system utilizing the following comparisons:

1) *Transference of Configuration*: The first comparison between the two environments is made in the transference of the configuration files from the production environment to the test bed. Configurations were accepted by the test bed devices and the test bed remained behaviorally consistent. Empirical testing demonstrated that minor variances such as firmware and operating system versions would prevent the deployment of and operation with these configuration files.

2) *State Consistency*: The second comparison is that of the state of the monitoring and control systems in both environments. During the period over which the historian data was collected from the production system (30 days), there was a corresponding state present in the production HMI, PLC, and historian. If the two systems match, the alerts, logs, and historian data should be consistent. In our test bed, we confirmed that the PLC displayed no communication alerts, and that the HMI displayed no alerts other than those it should due to the data being replayed.

3) *Consistency of Network Traffic*: We evaluated the consistency of the network traffic in the test bed relative to production. Each environment was sampled by means of a full PCAP recording via a passive network tap over a six-hour window. The data was summarized on a per-hour basis and the characteristics in Table I were used to compare the traffic from the two networks. In each environment the internal hour-to-hour comparisons have a coefficient of variance under 0.01% for most properties with the worst case being 0.03%. From this we assess that the traffic in both environments is stable and the data collected is sufficient for evaluation.

A comparison of the actual values for each of the characteristics within each environment is shown in Table I. Generally the values show the percentage difference is low ($< 1\%$). As can be seen, the last two values (TCP Frames/Bytes) show a significant difference. This difference is due to minor differences in the behavior of the VFD emulators we developed (see section 3) and their production counterparts. Specifically, while they both have a long-held-open TCP connection and pass the majority of their data via UDP, the production version resets the TCP connection periodically (resulting in additional TCP packets) while the emulator holds it open. Table II breaks down the percentage of traffic by type. TCP packets are less than 2% of the total; therefore, the difference in TCP traffic seen in Table I has little impact on the overall behavioral consistency of the network.

Overall, the consistency of the traffic between the two environments illustrates the negligible impact of the modifications to the PLC logic described above. It also provides a quantifiable measure of the delta between the two environments

TABLE I
CHARACTERIZATION OF THE NETWORK TRAFFIC IN THE PRODUCTION AND TEST BED ENVIRONMENTS

	Percentage Difference
Packet Count	0.72%
Data Size	0.88%
Avg Pkt Size	0.16%
Avg Pkt Rate	0.72%
Ethernet Frames	0.72%
Ethernet Bytes	0.88%
IP Frames	0.45%
IP Bytes	0.73%
UDP Frames	0.07%
UDP Bytes	0.04%
TCP Frames	28.68%
TCP Bytes	23.94%

TABLE II
UDP VS. TCP PACKET LOAD

	Production	Test Bed
UDP Packet Count	98.16%	98.67%
UDP Bytes	96.78%	97.52%
TCP Packet Count	1.84%	1.32%
TCP Bytes	3.22%	2.47%

and a clear understanding of how the results of an assessment performed in one environment may translate to the other.

VI. CONCLUSION

In this paper we provide a methodology for developing an ICS test bed that exhibits a high degree of realism with respect to its production counterpart. We then apply that approach to build a test bed for a system that supports the fastest supercomputer in the world. Finally we validate the system and discuss results showing the consistency of both sensor control and network behavior that are within 1% of production.

In future work we will examine the implications of adding physics-based models to increase the realism in sensor/actuator behavior while the system is under attack. Additionally, we will evaluate the appropriateness of using historian data as the emulation source for the sensors/actuators and the implications of gathering historian data at various sampling rates. Our interest also includes evaluating the differences between the VFD emulator and the production version. The above research activities support our primary objective in which we will test our hypothesis that in order to accurately assess a network-based intrusion detection device or similar network protection system, the test environment must closely resemble the target environment.

ACKNOWLEDGMENTS

Thank you, D. Grant, J. Carter, M. Rice and reviewers whose support and comments helped to ensure this document was both accurate and intelligible. This material is based on research sponsored by the Laboratory Directed Research

and Development Program of Oak Ridge National Laboratory, managed by UT-Battelle, LLC, for the U. S. Department of Energy. Additionally, this research used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725.

REFERENCES

- [1] R. Candell, K. A. Stouffer, and D. Anand, "A Cybersecurity Testbed for Industrial Control Systems," in *Proceedings of the 2014 Process Control and Safety Symposium*, Houston, TX, Oct. 2014.
- [2] J. Goh, S. Adepu, M. Tan, and Z. S. Lee, "Anomaly Detection in Cyber Physical Systems Using Recurrent Neural Networks," in *2017 IEEE 18th International Symposium on High Assurance Systems Engineering (HASE)*, Jan. 2017, pp. 140–145.
- [3] L. A. Maglaras, J. Jiang, and T. J. Cruz, "Combining ensemble methods and social network metrics for improving accuracy of OCSVM on intrusion detection in SCADA systems," *Journal of Information Security and Applications*, vol. 30, pp. 15–26, Oct. 2016.
- [4] S. S and P. W. D., "An enhanced optimization based algorithm for intrusion detection in SCADA network," *Computers & Security*, vol. 70, pp. 16–26, Sep. 2017.
- [5] M. Kalech, "Cyber-attack detection in SCADA systems using temporal pattern recognition techniques," *Computers & Security*, vol. 84, pp. 225–238, Jul. 2019.
- [6] Y. Yang, K. McLaughlin, S. Sezer, T. Littler, E. G. Im, B. Pranggono, and H. F. Wang, "Multiattribute SCADA-Specific Intrusion Detection System for Power Networks," *IEEE Transactions on Power Delivery*, vol. 29, no. 3, pp. 1092–1102, Jun. 2014.
- [7] D. Upadhyay and S. Sampalli, "SCADA (Supervisory Control and Data Acquisition) systems: Vulnerability assessment and security recommendations," *Computers & Security*, vol. 89, p. 101666, Feb. 2020.
- [8] H. Holm, M. Karresand, A. Vidström, and E. Westring, "A Survey of Industrial Control System Testbeds," in *Secure IT Systems*, ser. Lecture Notes in Computer Science, S. Buchegger and M. Dam, Eds. Cham: Springer International Publishing, 2015, pp. 11–26.
- [9] D. Formby, A. Walid, and R. Beyah, "A Case Study in Power Substation Network Dynamics," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 1, no. 1, pp. 19:1–19:24, Jun. 2017.
- [10] S. Lüders, "CERN tests reveal security flaws with industrial network devices," in *The Industrial Ethernet Book*, Nov. 2006, pp. 12–18.
- [11] Q. Qassim, N. Jamil, I. Z. Abidin, M. E. Rusli, S. Yussof, R. Ismail, F. Abdullah, N. Ja'afar, H. C. Hasan, and M. Daud, "A Survey of SCADA Testbed Implementation Approaches," *Indian Journal of Science and Technology*, vol. 10, no. 26, Jul. 2017.
- [12] "National SCADA Test Bed," <https://www.energy.gov/oe/technology-development/energy-delivery-systems-cybersecurity/national-scada-test-bed>.
- [13] H. G. Aghamolki, Z. Miao, and L. Fan, "A hardware-in-the-loop SCADA testbed," in *2015 North American Power Symposium (NAPS)*. Charlotte, NC, USA: IEEE, Oct. 2015, pp. 1–6.
- [14] I. Ahmed, V. Roussev, W. Johnson, S. Senthivel, and S. Sudhakaran, "A SCADA System Testbed for Cybersecurity and Forensic Research and Pedagogy," in *Proceedings of the 2nd Annual Industrial Control System Security Workshop*, ser. ICSS '16. Los Angeles, CA, USA: Association for Computing Machinery, Dec. 2016, pp. 1–9.
- [15] A. P. Mathur and N. O. Tippenhauer, "SWaT: A water treatment testbed for research and training on ICS security," in *2016 International Workshop on Cyber-Physical Systems for Smart Water Networks (CySWater)*, Apr. 2016, pp. 31–36.
- [16] "Summit," <https://www.olcf.ornl.gov/olcf-resources/compute-systems/summit/>.
- [17] "November 2019 — TOP500 Supercomputer Sites," <https://www.top500.org/lists/2019/11/>.